

Operaciones con cadenas de texto strings

Santiago Gallego

2026-05-07

Contenido

0.1 Ejercicio 1

Contexto: Has recibido una base de datos antigua del IGAC con los nombres de las áreas protegidas de Colombia. Los datos fueron ingresados manualmente a lo largo de los años, por lo que tienen serios problemas de formato (espacios extra, mayúsculas inconsistentes y errores de digitación). Necesitas estandarizar el nombre del Parque Nacional Natural Serranía de la Macarena para cruzarlo con el sistema actual.

1. Define una variable llamada `registro_crudo` que contenga exactamente este texto: " pnn seRRania de la mACaRena "

0.1.1 Python

```
registro_crudo = " pnn seRRania de la mACaRena " # Define el string con espacios y mayúsculas
```

0.1.2 R

```
registro_crudo <- " pnn seRRania de la mACaRena " # Define el string con espacios y mayúsculas
```

2. Utiliza el método nativo de tu lenguaje para eliminar los espacios en blanco al principio y al final del texto. Guarda el resultado en `texto_sin_espacios`.

0.1.3 Python

```
# 1. Eliminar espacios
texto_sin_espacios = registro_crudo.strip() # El método .strip() remueve espacios en ambos extremos
```

0.1.4 R

```
# 1. Eliminar espacios (trimws = trim white spaces)
texto_sin_espacios <- trimws(registro_crudo) # La función trimws limpia los espacios en blanco
```

3. Convierte el texto resultante a Formato Título (donde la primera letra de cada palabra es mayúscula) y guárdalo en `texto_titulo`.

0.1.5 Python

```
# 2. Formato Título
texto_titulo = texto_sin_espacios.title() # Pone la primera letra de cada palabra en mayúscula
```

0.1.6 R

```
# 2. Formato Título (R no tiene .title() nativo, usamos tools::toTitleCase o stringr)
# Usando una aproximación nativa:
texto_titulo <- tools::toTitleCase(tolower(texto_sin_espacios)) # Pasa a minúsculas y luego apl
```

4. El sistema oficial requiere que las siglas “PNN” estén en mayúscula sostenida. Utiliza la función de reemplazo para cambiar la subcadena “Pnn” por “PNN” dentro de tu texto_titulo. Guarda esto como registro_oficial.

0.1.7 Python

```
# 3. Reemplazar siglas
registro_oficial = texto_titulo.replace("Pnn", "PNN") # Busca "Pnn" y lo cambia por las siglas
```

0.1.8 R

```
# 3. Reemplazar siglas
registro_oficial <- gsub("Pnn", "PNN", texto_titulo) # Función de reemplazo global para corregir
```

5. Calcula la longitud (cantidad de caracteres) de tu registro_oficial.

0.1.9 Python

```
# 4. Longitud
longitud = len(registro_oficial) # La función len() cuenta la cantidad total de caracteres
```

0.1.10 R

```
# 4. Longitud
longitud <- nchar(registro_oficial) # La función nchar() cuenta el número de caracteres del str
```

6. Imprime un reporte final utilizando interpolación de cadenas que diga exactamente: “El área protegida limpia es: [AQUI_TU_VARIABLE] y su nombre tiene [AQUI_LONGITUD] caracteres.”

0.1.11 Python

```
# 5. Reporte con f-string
print(f"El área protegida limpia es: {registro_oficial} y su nombre tiene {longitud} caracteres")
```

El área protegida limpia es: PNN Serrania De La Macarena y su nombre tiene 28 caracteres.

0.1.12 R

```
# 5. Reporte con interpolación
cat(sprintf("El área protegida limpia es: %s y su nombre tiene %d caracteres.\n", registro_oficial, longitud))
```

El área protegida limpia es: PNN serrania De La Macarena y su nombre tiene 32 caracteres.

0.2 Ejercicio 2

Contexto: Un sensor sísmico ubicado en el Volcán Galeras (Nariño) envía las alertas a través de mensajes de texto planos. Tu objetivo es extraer las coordenadas de ese mensaje de texto, convertirlas a formato numérico, y ensamblar un punto espacial estándar (Well-Known Text) para insertarlo en la base de datos PostGIS del Servicio Geológico.

1. Define la variable `mensaje_sensor` con el siguiente texto: “ALERTA_SISMICA:1.221,-77.359:PROFUNDIDAD_5KM”

0.2.1 Python

```
mensaje_sensor = "ALERTA_SISMICA:1.221,-77.359:PROFUNDIDAD_5KM" # String de entrada con delimitador
```

0.2.2 R

```
mensaje_sensor <- "ALERTA_SISMICA:1.221,-77.359:PROFUNDIDAD_5KM" # String de entrada con delimitador
```

2. Utiliza la función de separación (`Split`) usando los dos puntos (`:`) como delimitador. Extrae únicamente el bloque central que contiene las coordenadas y guárdalo en la variable `bloque_coords`.

0.2.3 Python

```
# 1 y 2. Split por ":" y extraer bloque central
bloque_coords = mensaje_sensor.split(":")[1] # Divide por ":" y toma el segundo elemento (índice 1)
```

0.2.4 R

```
# 1 y 2. Split por ":" y extraer bloque central
bloque_coords <- strsplit(mensaje_sensor, ":")[[1]][2] # Divide la cadena y extrae el segundo c
```

3. Vuelve a aplicar la función de separación (Split), esta vez sobre bloque_coords, usando la coma (,) como delimitador para separar la latitud de la longitud.

0.2.5 Python

```
# 3. Split por ","
partes = bloque_coords.split(",") # Separa el bloque de coordenadas en dos partes usando la coma
```

0.2.6 R

```
# 3. Split por ","
partes <- strsplit(bloque_coords, ",")[[1]] # Separa el bloque en un vector de dos elementos (I
```

4. Las coordenadas extraídas siguen siendo texto. Conviértelas a formato numérico decimal (Float) y guárdalas en las variables latitud y longitud.

0.2.7 Python

```
# 4. Convertir a Float
latitud = float(partes[0]) # Convierte la primera parte de texto a número decimal
longitud = float(partes[1]) # Convierte la segunda parte de texto a número decimal
```

0.2.8 R

```
# 4. Convertir a numérico
latitud <- as.numeric(partes[1]) # Convierte el primer elemento del vector a numérico
longitud <- as.numeric(partes[2]) # Convierte el segundo elemento del vector a numérico
```

5. La base de datos requiere una precisión estricta. Utilizando las herramientas de formateo de tu lenguaje, construye una cadena de texto en formato WKT (POINT(lon lat)) asegurando que tanto la latitud como la longitud tengan **exactamente 4 decimales**. Guarda esto en punto_wkt. (Nota: Recuerda que WKT usa el orden Longitud Latitud, separados por un espacio, no por coma).

0.2.9 Python

```
# 5. Construir WKT con 4 decimales
punto_wkt = f"POINT({longitud:.4f} {latitud:.4f})" # Crea el formato espacial fijando 4 decimales
```

0.2.10 R

```
# 5. Construir WKT (formatC asegura los 4 decimales)
lon_f <- formatC(longitud, digits = 4, format = "f") # Formatea la longitud a 4 decimales fijos
lat_f <- formatC(latitud, digits = 4, format = "f") # Formatea la latitud a 4 decimales fijos
punto_wkt <- paste0("POINT(", lon_f, " ", lat_f, ")") # Une las piezas para formar el texto WKT
```

6. El sensor reportó una anomalía térmica que afectó un área de 1250000 metros cuadrados. Formatea este número agregando un separador de miles (coma o punto) para que sea legible en el reporte final.

0.2.11 Python

```
# 6. Formatear área con separador de miles
area = 1250000 # Valor numérico del área
area_formateada = f"{area:,}" # Usa la coma como separador de miles automáticamente
```

0.2.12 R

```
# 6. Formatear área con separador de miles
area <- 1250000 # Valor numérico del área
area_formateada <- format(area, big.mark = ",") # Aplica formato con coma para separar miles
```

7. Imprime el WKT generado y el área afectada formateada.

0.2.13 Python

```
# 7. Imprimir
print(f"WKT: {punto_wkt}") # Muestra el punto en formato Well-Known Text
```

WKT: POINT(-77.3590 1.2210)

```
print(f"Área afectada: {area_formateada} m²") # Muestra el área con formato de lectura fácil
```

Área afectada: 1,250,000 m²